COMP4127/COMP7850

Information Security

Web Security

Reading: OWASP Top 10 - 2021 The Ten Most Critical Web Application Security Risks

Learning Outcome

After this lecture you should be able to:

- Understand ten commonly seen web security issue;
- Name some mechanism to defense or avoid these security attacks/issues;
- Name some best practices for developer in software development.

While reading this lecture note, please also run the following and open the webpage on http://localhost:8000

```
docker run --rm -p 8000:3000 -d bkimminich/juice-shop
```

OWASP

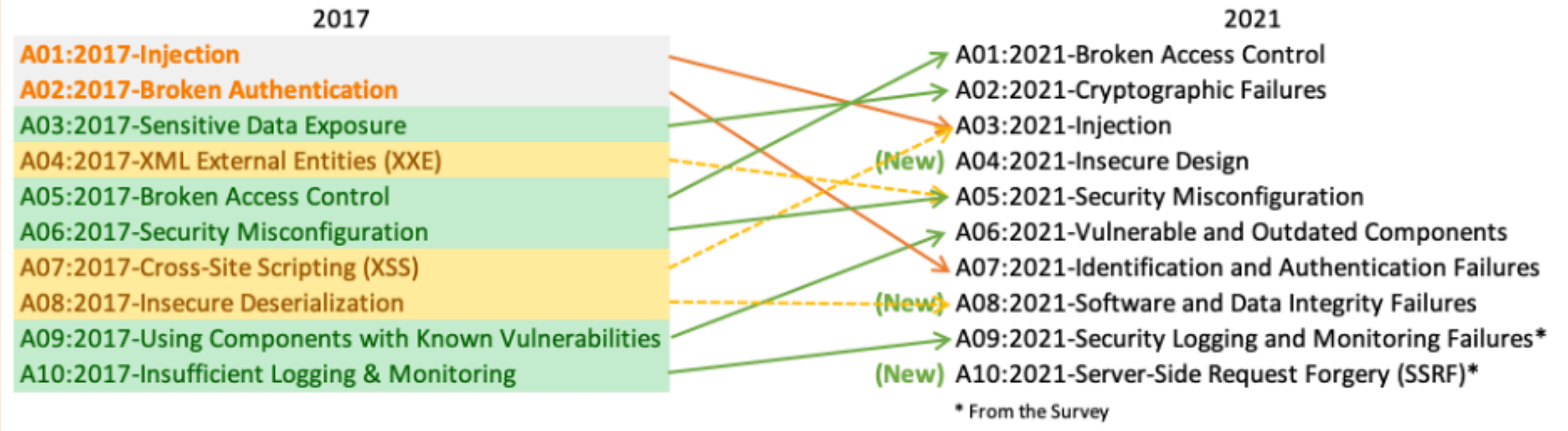
- The Open Web Application Security Project (OWASP) is a nonprofit foundation that works to improve the security of software. Through community-led open-source software projects, hundreds of local chapters worldwide, tens of thousands of members, and leading educational and training conferences, the OWASP Foundation is the source for developers and technologists to secure the web.

优先考虑 流行

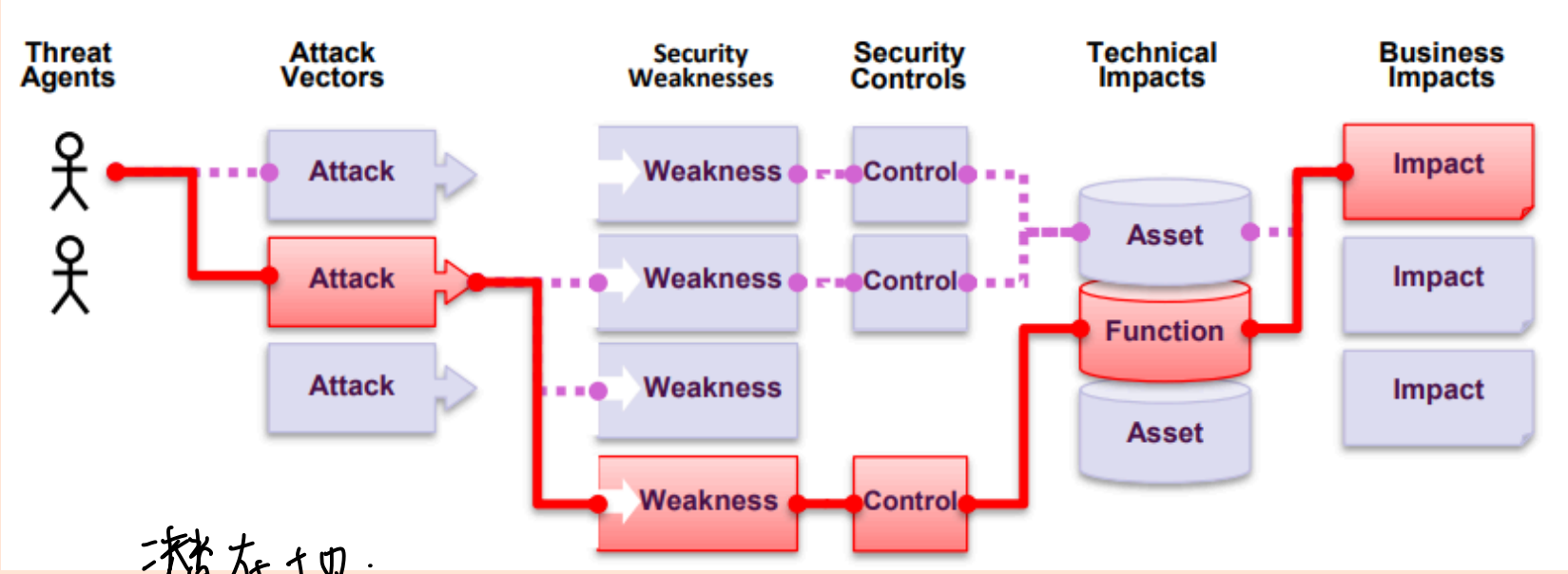
- The Top 10 items are selected and prioritized according to this prevalence data, in combination with consensus estimates of exploitability, detectability, and impact.

察觉

Changes from 2017 to 2021



Application Security Risk



潜在地.

- Attackers can potentially use many different paths through your application to do harm to your business or organization.
- Each of these paths represents a risk that may, or may not, be serious enough to warrant attention.

正当授权

OWASP Risk Model

OWASP define a risk rating system to assist in calculating the likelihood and impact of any given risk.

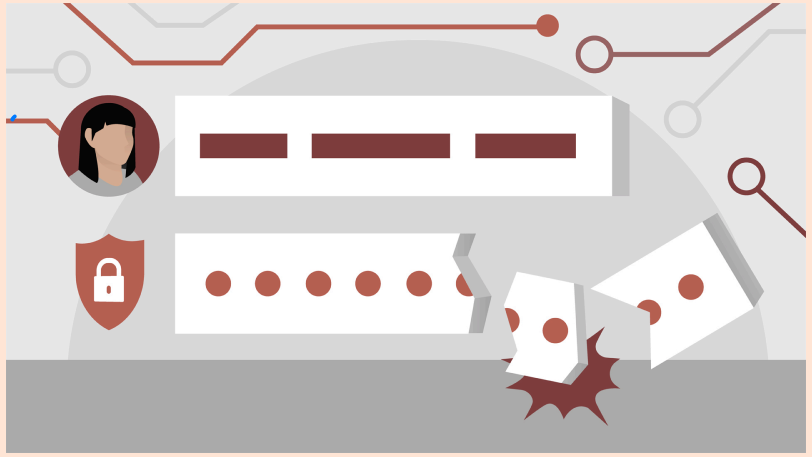
Threat Agents	Exploitability	Weakness Prevalence	Weakness Detectability	Technical Impacts	Business Impacts
Appli-cation Specific	Easy: 3	Widespread: 3	Easy: 3	Severe: 3	Business Specific
	Average: 2	Common: 2	Average: 2	Moderate: 2	
	Difficult: 1	Uncommon: 1	Difficult: 1	Minor: 1	

The Top-10

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server Side Request Forgery (SSRF)

Broken Access Control

- Restrictions on what authenticated users are allowed to do are often not properly enforced. 实施
- Attackers can exploit these flaws to access unauthorized functionality and/or data, such as access other users' accounts, view sensitive files, modify other users' data, change access rights, etc.



Risks

違反 特权

- Violation of the principle of least privilege or deny by default (i.e. access is available to anyone).
- Bypassing access control checks by modifying the URL (parameter tampering 篡改 or force browsing), or the HTML page.
- Permitting viewing or editing someone else's account, by providing its unique identifier (insecure direct object references)
- Accessing API with missing access controls for POST, PUT and DELETE.
- **Force browsing** to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

Example 1

The application uses unverified data in a SQL call that is accessing account information:

```
pstmt.setString(1, request.getParameter("acct"));
ResultSet results = pstmt.executeQuery( );
```

An attacker simply modifies the browser's 'acct' parameter to send whatever account number they want. If not correctly verified, the attacker can access any user's account.

only one user infor can be accessed.

Example 2

An attacker simply forces browses to target URLs. Admin rights are required for access to the admin page.

```
https://example.com/app/getappInfo
https://example.com/app/admin_getappInfo
```

If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.

瑕疵.

JuiceShop - Forgotten Developer Backup

Knowledge background - URL Encode

Data is submitted to a server via a `GET` or `PUT` command uses the following syntax to embed the data into a URL:

e.g. `GET`: http://www.example.host/mypage.jsp?username=kevin&password=abcd

URL may only contains ASCII characters, all special character needs to be submitted will need to be encoded by **URL Encoding**, e.g.:

Hello Günter ➡ `Hello%20G%C3%BCnter`

In URL encoding two hex values followed by a `%` symbol represents a character to be submitted.

ASCII	null	space	%	&	<	>
URL-Encoding	%00	%20	%25	%26	%3C	%3E

Explaining the problem - URL Encode

Consider the following snipper in a server

```
String fn = request.getParameter("fn");
if (fn.endsWith(".db")) {
  File f = new File(fn);
  //read the contents of "f" file
  ...
}
```

Access with the string `http://server/mypage.jsp?fn=serverlogs.txt%00.db` will be able to fool the program as:

- `fn` looks like a string endsWith ".db".
- It attempts to open this filename and will ignore the text after the null string.

In Juice-shop example, `%2500` will be translated to `%00` after one url decode and is translated to null after the second decoding.

Defenses

Access control is only effective in trusted server-side code or server-less API, where the attacker cannot modify the access control check or metadata.

- Except for public resources, **deny by default**.
- Implement access control mechanisms once and re-use them throughout the application.
- Model access controls should enforce record ownership (read/write by owner only).
- Unique application business limit requirements should be enforced by domain models.
- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.
- Log access control failures, alert admins when appropriate (e.g., repeated failures).

  警惕
- Limit APIs access to minimize the harm from automated attack tooling.

Cryptographic Failures

- Many web applications and APIs do not properly protect sensitive data, such as financial, healthcare, and PII.
- Attackers may steal or modify such **weakly protected** data to conduct credit card fraud, identity theft, or other crimes.

暴露

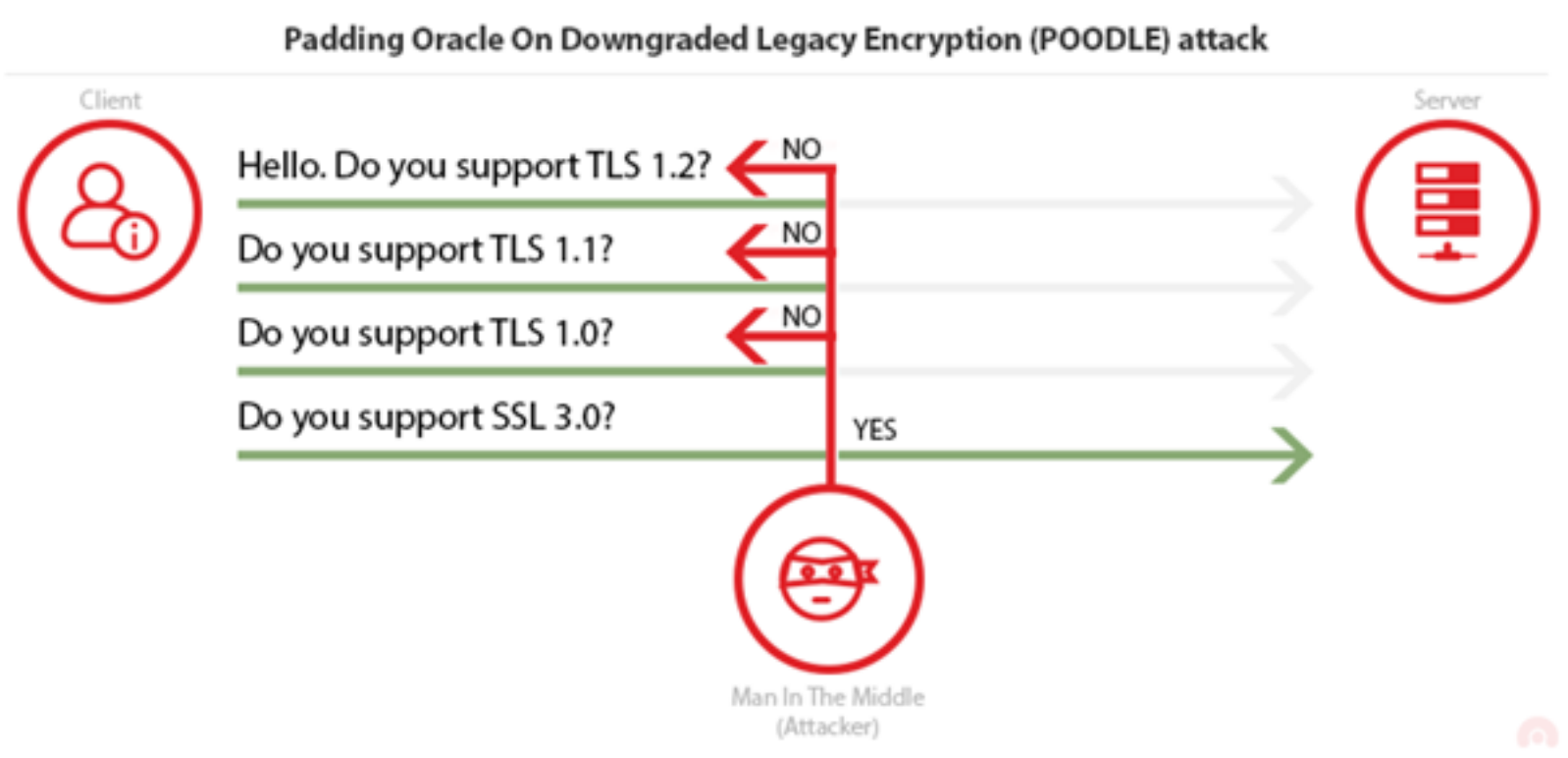
Sensitive data may be compromised without extra protection, such as encryption at rest or in transit, and requires special precautions when exchanged with the browser.

預防措施

Risks

- Is any data transmitted in clear text? e.g HTTP, SMTP, and FTP.
- Is sensitive data stored in clear text, including backups?
- Are any old or weak cryptographic algorithms used? Hard-code IV?
- Are default crypto keys in use, weak crypto keys generated or re-used?
- Is encryption not enforced, e.g. are any user agent (browser) security directives or headers missing?
- Does the user agent (e.g. app, mail client) not verify if the received server certificate is valid?

Examples - POODLE attack



A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g., at an insecure wireless network), **downgrades connections** from TLS to insecure protocol, intercepts requests, and steals the user's session cookie. → 降級

img credit: https://www.acunetix.com/blog/articles/tls-vulnerabilities-attacks-final-part/

Defenses

- Classify data processed, stored, or transmitted by an application.
- Apply controls as per the classification.
- Don't store sensitive data unnecessarily (discard/tokenize/truncate).
- Make sure to encrypt all sensitive data at rest. 强制性.
- Use up-to-date and standard algorithms and protocols
- Use proper key management.
- Encrypt all data in transit with secure protocols.
- Enforce encryption using directives like HTTP Strict Transport Security (**HSTS**).
- Disable caching for responses that contain sensitive data. 抓取、储存
- Store passwords using strong adaptive and **salted hashing** functions with a work factor (delay factor), such as Argon2, scrypt, bcrypt, or PBKDF2.
- Verify independently the effectiveness of configuration and settings.

Injection

Injection flaws occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization.

Different type of injection

- SQL Injection
- XXE
- XSS

Risks

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without contextaware escaping are used directly in the interpreter.
- Hostile data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Hostile data is directly used or concatenated, such that the SQL or command contains both structure and hostile data in dynamic queries, commands, or stored procedures.

Example Juice Shop: Login / CTF SQL injection

Cross-Site Scripting (XSS)

- Occur whenever an application **includes untrusted data in a web page** without proper validation or escaping, or updates an existing web page with user-supplied data using a browser API that can create HTML or JavaScript.
- Allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites.
- Modern browser would stop XSS in many cases, e.g. embedding inside a `<div>` normally does not work.
- `<iframe>` is very vulnerable against XSS.

脆弱的

Examples

- JuiceShop - extra payload
- `<script>alert('evil code')</script>`
- `<img src=x onerror="alert('Pop-up window via stored XSS');"`

Defenses

- Use safe API (e.g. `prepareStatement`)
- Use ORM (Object Relational Mapping) tools
- Sanitize inputs - character and string escaping: e.g. converting `<` symbol to `<` so that no script can be embedded easily.

> `<script>alert('testing')</script>` will be executed, but `<script>alert('testing')</script>` will not.

- Use a template system with context-aware auto-escaping. Some development framework will automatically escape user input in order to prevent any scripts embedded in the input from executing.

Insecure Design

There is a difference between insecure design and insecure implementation.

- A secure design can still have implementation defects leading to vulnerabilities that may be exploited.
- An insecure design cannot be fixed by a perfect implementation as by definition, needed security controls were never created to defend against specific attacks.

One of the factors that contribute to insecure design is the lack of business risk profiling inherent in the software or system being developed, and thus the failure to determine what level of security design is required.

概述内在的

Example 1

热衷者

拍卖

零售

A retail chain's e-commerce website does not have protection against bots run by scalpers buying high-end video cards to resell auction websites. This creates 持久的. terrible publicity for the video card makers and retail chain owners and enduring bad blood with enthusiasts who cannot obtain these cards at any price. Careful anti-bot design and domain logic rules, such as purchases made within a few seconds of availability, might identify inauthentic purchases and rejected such transactions.

发现不可信赖的购买

Example 2

A credential recovery workflow might include “questions and answers,” which is prohibited by NIST 800-63b, the OWASP ASVS, and the OWASP Top 10. Questions and answers cannot be trusted as evidence of identity as more than one person can know the answers, which is why they are prohibited. Such code should be removed and replaced with a more secure design.

Mitigation 强解

- Use multiple security questions;
- Let the user select their own security questions or provide them with choices of questions that are not generic.
- E-mail the temporary password to the registered e-mail address.

Defense

- Use **secure development lifecycle** with security and privacy-related controls
- Use a library of secure design patterns or paved road ready to use components
- Use threat modeling for critical authentication, access control, business logic, and key flows
- Integrate security language and controls into user stories
- Integrate plausibility checks at each tier of your application (from frontend to backend)
- Write unit and integration tests to validate that all critical flows.
- Compile use-cases and misuse-cases for each tier of your application.

误用

Security Misconfiguration

哪些東西

Security misconfiguration is the most commonly seen issue. This is commonly a result of insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information. Not only must all operating systems, frameworks, libraries, and applications be securely configured, but they must be patched and upgraded in a timely fashion.

修补.

Risks

- Missing appropriate security hardening across any part of the application stack, or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g. unnecessary ports, services, pages, accounts, or privileges).
- Default accounts and their passwords still enabled and unchanged.
- Error handling reveals stack traces or other overly informative error messages to users.
- For upgraded systems, latest security features are disabled or not configured securely.
- The security settings in the application servers, application frameworks (e.g. Struts, Spring, ASP.NET), libraries, databases, etc. not set to secure values.

Example 1

A user leave its router admin password unchanged. Attacker can attach to the network (if open or leaving WiFi password unchanged) and take full control of the router.

Step 1

Open web browser and type LAN IP address of the wireless ADSL router in the address bar, and press Enter. The default IP address of TP-Link ADSL router is 192.168.1.1.

Address http://192.168.1.1/ Go

Please refer to: What should I do if I can't log into web-based interface of my TP-Link device?

Step 2

Type the username and password in the login page. The default username and password are both admin in lowercase.

img credit: Linksys

Example 2

The application server's configuration allows detailed error messages, e.g., stack traces, to be returned to users. This potentially exposes sensitive information or underlying flaws such as component versions that are known to be vulnerable.

Deploy

```
▶ Run git push -f ***git.heroku.com/$HEROKU_APP_NAME.git main
remote: Compressing source files... done.
remote: Building source:
remote:
remote: !       Your account has reached its concurrent builds limit.
remote:
To https://git.heroku.com/comp7940-21456291.git
 ! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to 'https://git.heroku.com/comp7940-21456291.git'
Error: Process completed with exit code 1.
```

A good practice in logging - the server key was masked (***) in the log

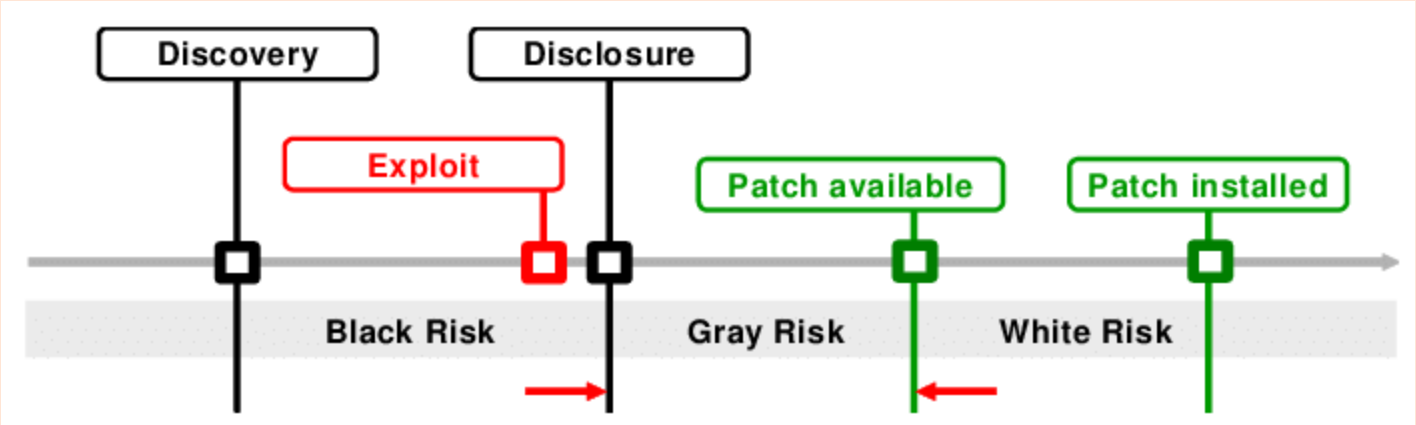
img credit: anonymous student from COMP7940.

Defense

- Development, QA, and production environments should all be configured identically, with different credentials used in each environment.
- A minimal platform without any unnecessary features, components, documentation, and samples. Remove or do not install unused features and frameworks.
- A task to review and update the configurations appropriate to all security notes, updates, and patches as part of the patch management process (see A06:2021-Vulnerable and Outdated Components). Review cloud storage permissions (e.g., S3 bucket permissions).
- A segmented application architecture provides effective and secure separation between components or tenants, with segmentation, containerization, or cloud security groups (ACLs).
- Sending security directives to clients, e.g., Security Headers.
- An automated process to verify the effectiveness of the configurations and settings in all environments.

Vulnerable and Outdated Components

A system using vulnerable or outdated component has severe security risk.



Lifecycle of a vulnerability

- **Discovery-Time**: the earliest date that the vulnerability is discovered
- **Exploit-Time**: the earliest date that an exploit is available.
- **Disclosure-Time**: the earliest date that the vulnerability is publicly known
- **Patch-Time**: the date that the patch is released.

补丁

ref: Frei, Stefan & Tellenbach, Bernhard & Plattner, Bernhard. (2008). 0-Day Patch Exposing Vendors (In)security Performance.

Lifecycle of a vulnerability

- **Black Risk**: Between discovery to disclosure, only a closed group is aware of the vulnerability. It can be either:
  黑客
  - Hacker groups to exploit the vulnerability;
  - Researchers/Vendors to prepare a fix for the identified vulnerability.
- **Gray Risk**: Between disclosure to patch, users are exposed at the highest risk where more hackers can exploit the vulnerability.
  卖主
  - Vendors work on patch and might implement a workaround until a patch is available.
    变通办法.
- **White Risk**: Between patch available and patch is actually installed.
  - User's fault!!!

Zero day and N day

Zero-day attacks

?

- Zero-day attack describes a situation that a software vulnerability is discovered and vendor has virtually no time to patch the vulnerability.
- Attacker exploits the vulnerability by writing scripts/bots.

N-day attacks

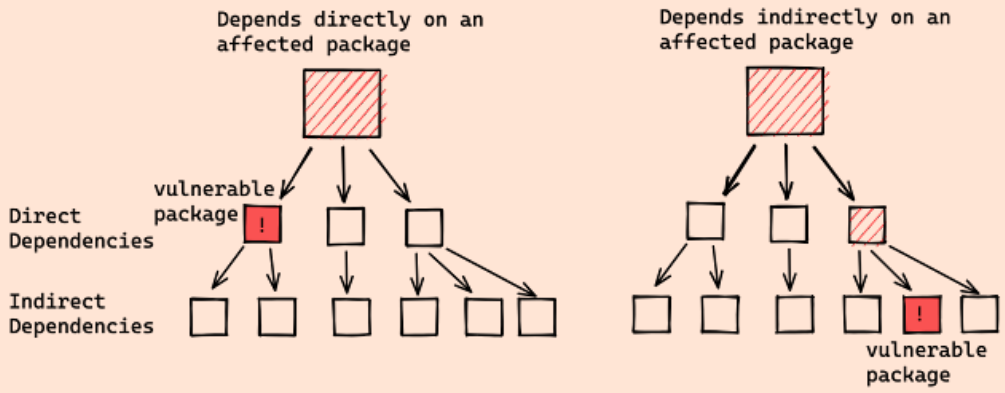
- An N-day exploited vulnerability that has a patch already.

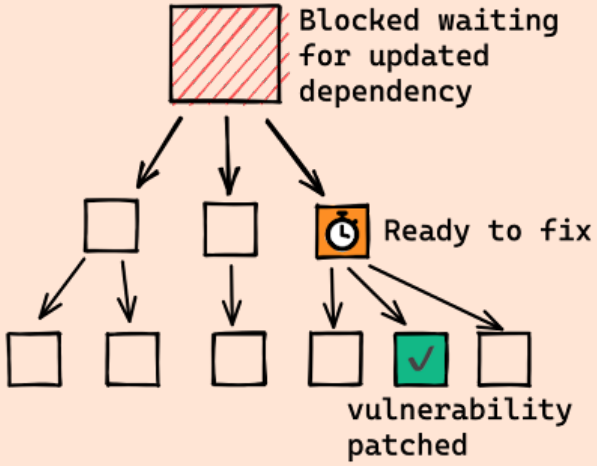
Example 1

A user is unwilling to perform system update because of various reasons: work interruption, risk of incompatible of existing software, or simply lazy. Sometimes the user simply does not aware there is system update available.

Example 2 - Log4j

On December 9, 2021, security researchers discovered a flaw in the code of a software library used for logging. This flaw in Log4j is estimated to be present in over 100 million instances globally.





Most artifacts that depend on log4j do so indirectly. The deeper the vulnerability is in a dependency chain, the more steps are required for it to be fixed.

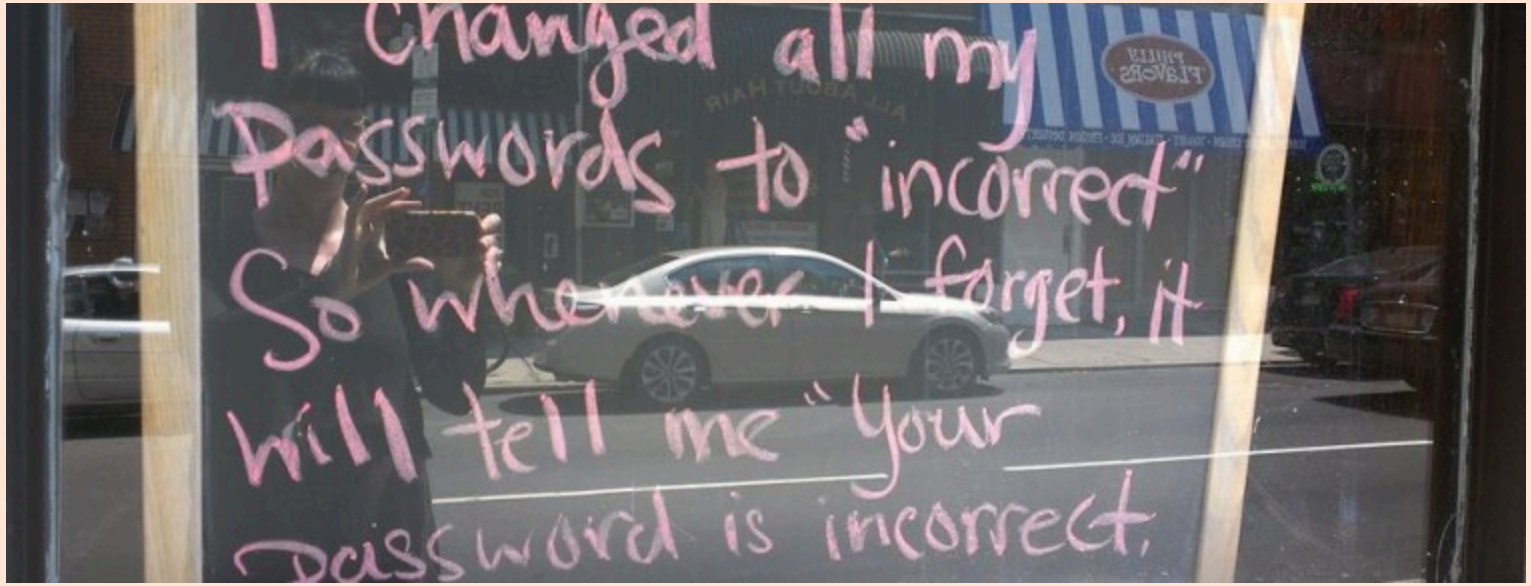
ref: CISECURITY.org, Googleblog.com

Defenses

- Remove unused dependencies, unnecessary features, components, files, and documentation.
- Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies using tools like versions, OWASP Dependency Check, retire.js, etc.
- Continuously monitor sources like Common Vulnerability and Exposures (CVE) and National Vulnerability Database (NVD) for vulnerabilities in the components.
- Use software composition analysis tools to automate the process.
- Subscribe to email alerts for security vulnerabilities related to components you use.
- Only obtain components from official sources over secure links.

Identification and Authentication Failures

Application functions related to authentication and session management are often implemented incorrectly, allowing attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users' identities temporarily or permanently.

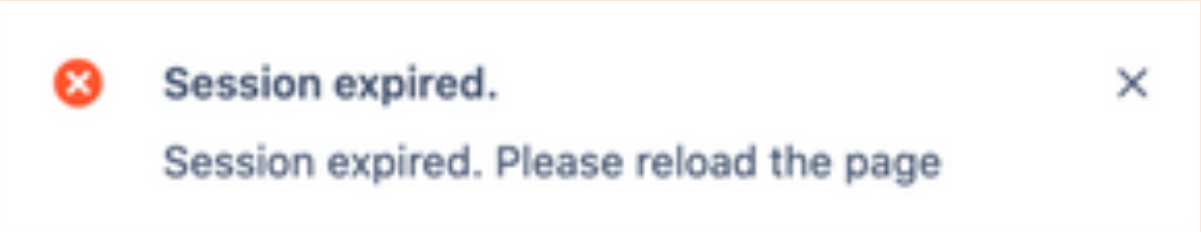


img credit: https://pressupinc.com/blog/2014/04/write-passwords-never-reuse/

Examples

Scenarios 1

Application session timeouts aren't set properly. A user uses a public computer to access an application. Instead of selecting "logout" the user simply closes the browser tab and walks away. An attacker uses the same browser an hour later, and the user is still authenticated.



Scenarios 2

Use of weak password leading to vulnerability of online dictionary attacks.

Risks

证件 填充

- Permits automated attacks such as credential stuffing, where the attacker has a list of valid usernames and passwords.
- Permits brute force or other automated attacks.
- Permits default, weak, or well-known passwords, such as "Password1" or "admin/admin".
- Uses weak or ineffective credential recovery and forgot password processes, such as "knowledge-based answers",which cannot be made safe.
- Uses plain text, encrypted, or weakly hashed passwords.
- Has missing 缺席 or ineffective multi-factor authentication.
- Exposes Session IDs in the URL (e.g., URL rewriting). ?
- Does not rotate Session IDs after successful login.

Defenses

- Multi-Factor Authentication
  - Check client's IP address, location
- CAPTCHA
- Device Fingerprinting
  - OS/Browser/Language/Screen resolution/fonts/browser plugins
- Notify users about unusual security events
- Do not ship or deploy with any default credentials, particularly for admin users. 运送 部署.
- Implement weak password checks.

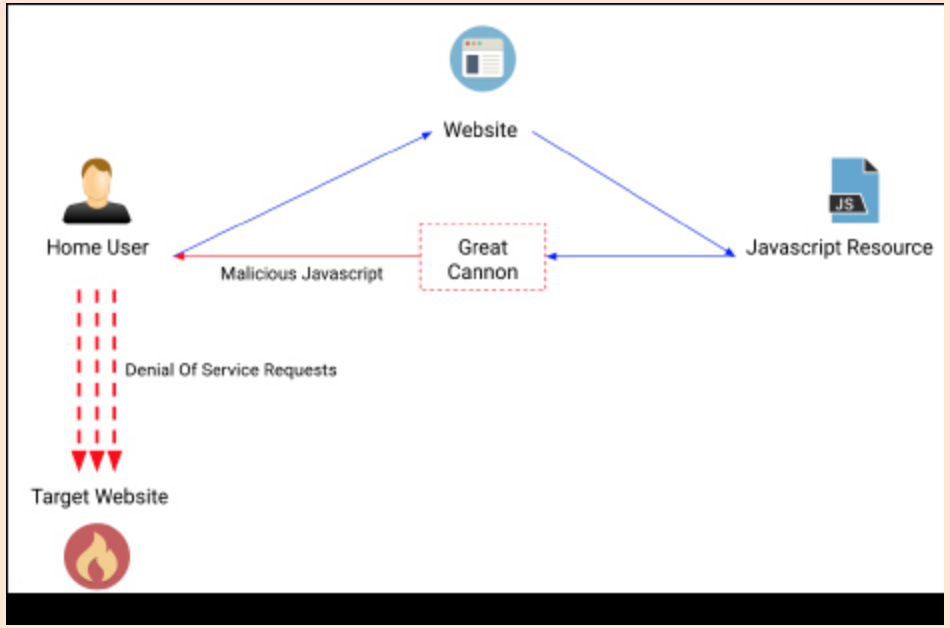
Software and Data Integrity Failures

- Relate to code and infrastructure that does not protect against integrity violations. 违反
- Software components are used without verification.

Example

嵌入式

Application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs). An insecure CI/CD pipeline can introduce the potential for unauthorized access, malicious code, or system compromise. Lastly, many applications now include auto-update functionality, where updates are downloaded without sufficient integrity verification and applied to the previously trusted application. Attackers could potentially upload their own updates to be distributed and run on all installations. Another example is where objects or data are encoded or serialized into a structure that an attacker can see and modify is vulnerable to insecure deserialization.



Further Reading: [The Great Cannon]

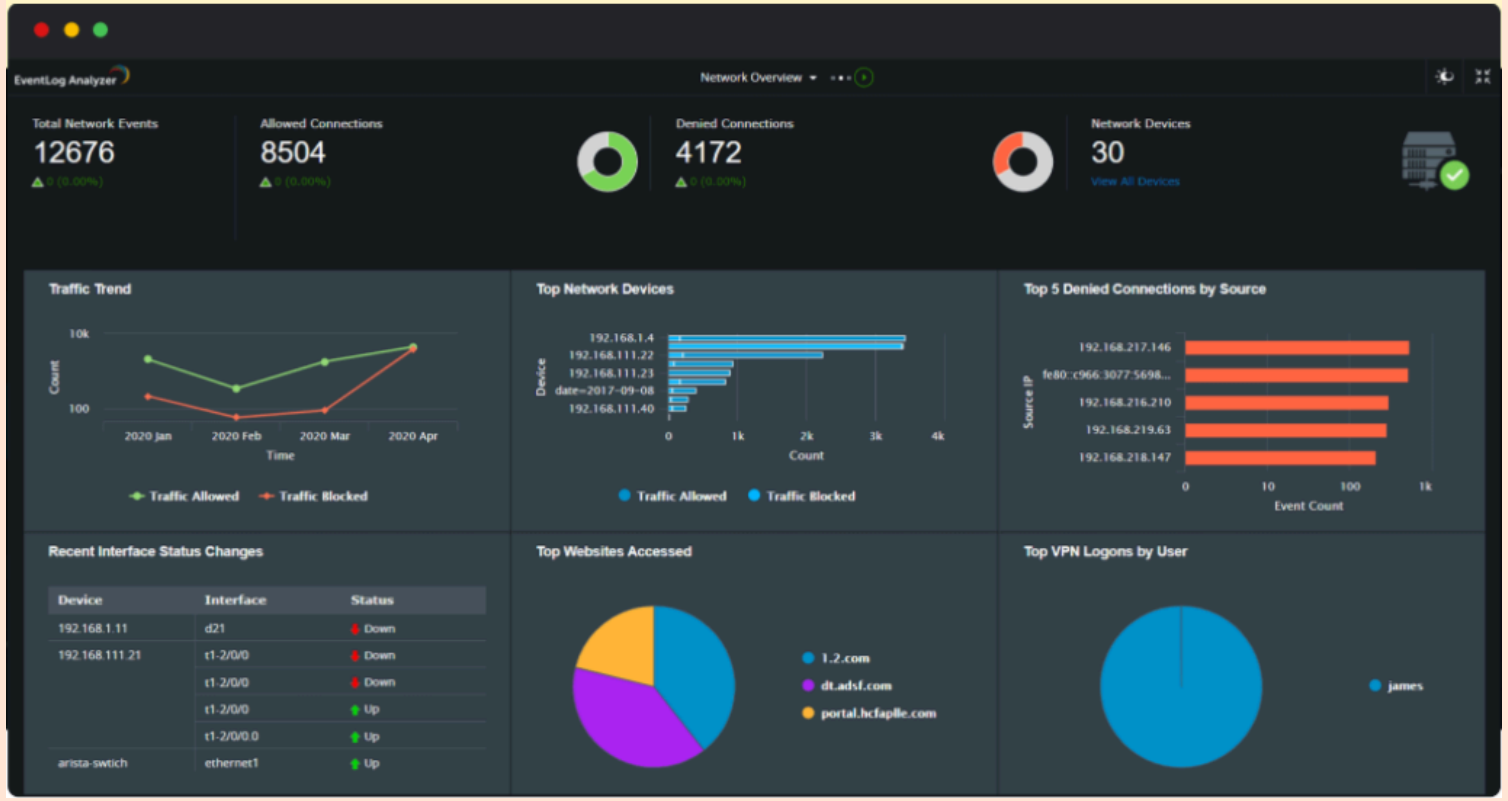
Defenses

- Use digital signatures or similar mechanisms to verify the software.
- Consume libraries and dependencies, such as npm or Maven, from trusted repositories.
- Use software supply chain security tool, such as OWASP Dependency Check or OWASP CycloneDX.
- A review process for code and configuration changes.
- Unsigned or unencrypted serialized data is not sent to untrusted clients.

Security Logging and Monitoring Failures

破坏

- Without logging and monitoring, breaches cannot be detected.
- Insufficient logging, detection, monitoring, and active response occurs any time.



ref: https://www.manageengine.com/products/eventlog/index-new.html

Example

A children's health plan provider's website operator couldn't detect a breach 漏洞 due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.

img credit: https://www.leap.expert/data-leakage-the-risk-the-reality-and-messaging-channels/

Defense

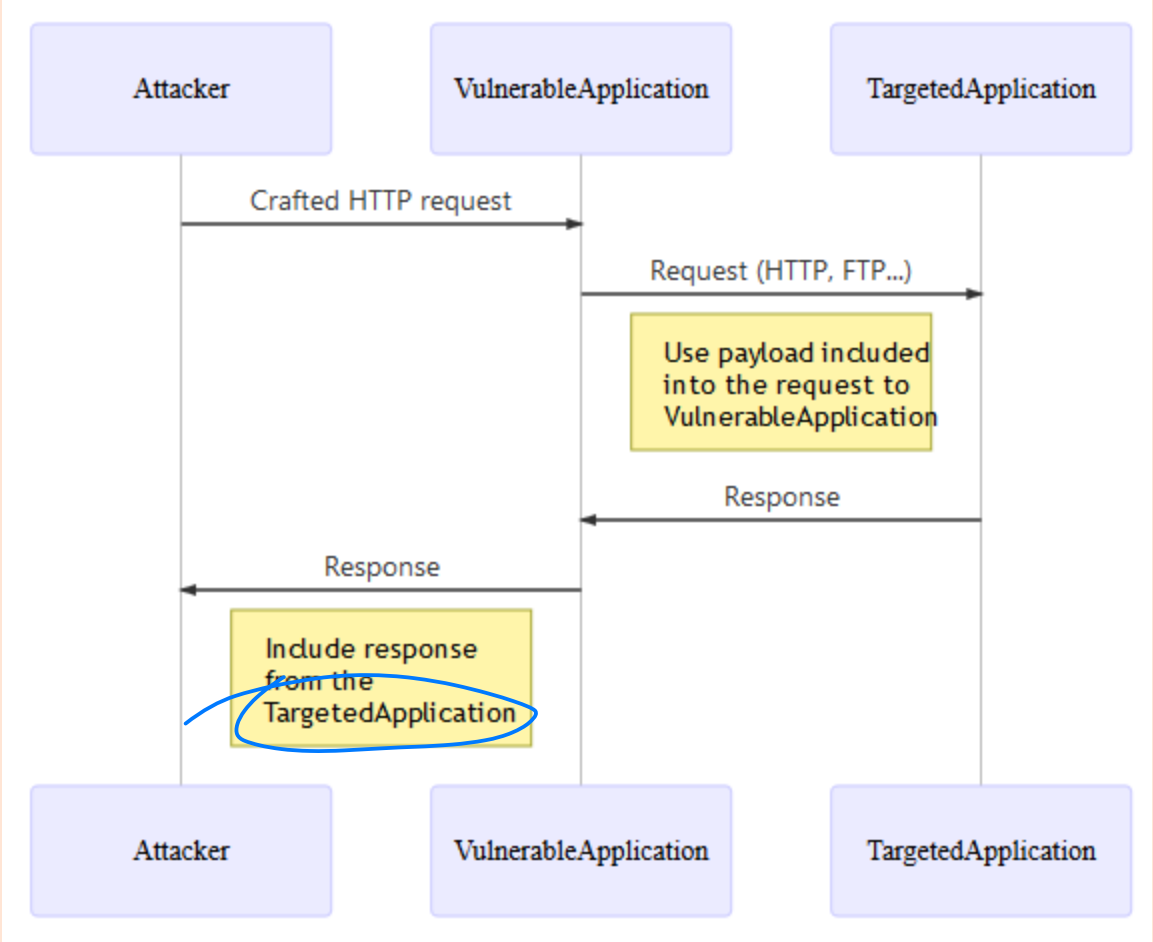
- All login, access control, and server-side input validation failures are logged with sufficient user context.
- Logs are generated in a format that log management solutions can easily consume.
- Logs are encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- High-value transactions have an audit trail with integrity controls to prevent tampering or deletion.
- DevSecOps teams should establish effective monitoring and alerting such that suspicious activities are detected and responded to quickly.
- Establish incident response and recovery plan.

Server-Side Request Forger (SSRF)

伪造、

SSRF flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).

强制



SSRF

展示

One of the cause of the flaw is the mishandling of URLs, as showcased in the following examples:

化身

- Image on an external server (e.g. user enters image URL of their avatar for the application to download and use).
- Custom WebHook (users have to specify Webhook handlers or Callback URLs).
- Internal requests to interact with another service to serve a specific functionality. Most of the times, user data is sent along to be processed, and if poorly handled, can perform specific injection attacks (e.g. XXE).

XML External Entities (XXE)

- An **XML External Entity attack** is a type of attack against an application that parses XML input. 符合语法
- **When XML input containing a reference to an external entity is processed by a weakly configured XML parser**.
- Lead to the disclosure of confidential data, denial of service, server side request forgery, port scanning from the perspective of the machine where the parser is located, and other system impacts.

For example, suppose a shopping application checks for the stock 库存 level of a product by submitting the following XML to the server:
A normal XML submission from client may looks like

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck><productId>381</productId></stockCheck>
```

XXE Examples (con't)

To perform an XXE injection attack that retrieves an arbitrary file from the server's filesystem, you need to modify the submitted XML in two ways:

- Introduce (or edit) a DOCTYPE element that defines an external entity containing the path to the file.
- Edit a data value in the XML that is returned in the application's response, to make use of the defined external entity.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<stockCheck><productId>&xxe;</productId></stockCheck>
```

This causes the application's response to include the contents of the file:

```
Invalid product ID: root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

can not access

SSRF vs Injection

While XXE or SQL injection may also be classified as both categories, SSRF focuses on the **vulnerable application** to impose 强制执行 mechanism to sanitize inputs or restrict the server request by configuration (i.e. request can only made to destined servers.)

Defenses

From Network layer

- Segment remote resource access functionality in separate networks to reduce the impact of SSRF
- Enforce "deny by default" firewall policies or network access control rules to block all but essential intranet traffic.

From Application layer:

- Sanitize and validate all client-supplied input data
- Enforce the URL schema, port, and destination with a positive allow list
- Do not send raw responses to clients

For Developers

1. Application Security Requirements

- To produce a secure web application, you must define what secure means for that application.
- OWASP recommends you use the OWASP Application Security Verification Standard (ASVS) as a guide for setting the security requirements for your application(s).

2. Application Security Architecture

加装

- Rather than retrofitting security into your applications and APIs, it is far more cost effective to design the security in from the start.
- OWASP recommends the **OWASP Prevention Cheat Sheets** as a good starting point for guidance on how to design security in from the beginning.

For Developers (con't)

3. Standard Security Controls

- Building strong and usable security controls is difficult. Using a set of standard security controls radically simplifies the development of secure applications and APIs.
- The OWASP Proactive Controls is a good starting point for developers, and many modern frameworks now come with standard and effective security controls for authorization, validation, CSRF prevention, etc.

4. Secure Development Lifecycle

- To improve the process your organization follows when building applications and APIs, OWASP recommends the OWASP Software Assurance Maturity 成熟. Model (SAMM). This model helps organizations formulate and implement a strategy for software security that is tailored to the specific risks facing their organization.

可做的.

For Developers (con't)

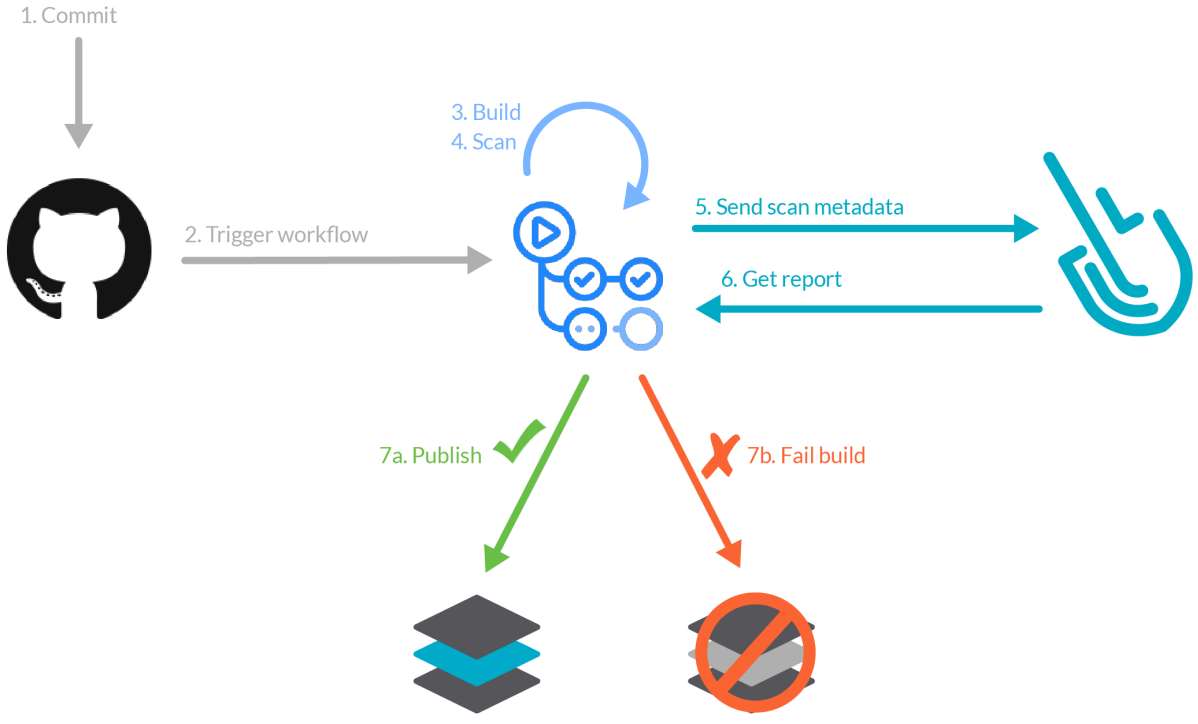
5. Application Security Education

- The OWASP Education Project provides training materials to help educate developers on web application security.
- For hands-on learning about vulnerabilities, e.g, OWASP Juice Shop Project.

Continuous Application Security Testing

The goal of application security testing is to provide the evidence of the security you intended to build is actually present, correctly implemented, and used everywhere it is supposed to be.

Modern software development requires continuous application security testing across the entire software development lifecycle. Look to enhance existing development pipelines with security automation that doesn't slow development.



e.g.

https://github.com/marketplace/actions/synopsys-intelligent-security-scan

https://github.com/marketplace/actions/aqua-security-trivy